ASTRA

Deploying ASTRA securely in organisations

Deployment options, whether to ship an .exe, the security mechanisms (HTTPS and more), and how to keep updating from GitHub safely.

Report based on the current code · Version 1.0 · 23 October 2026

1. The short answer

Do not ship ASTRA as an .exe. Ship it as a **container bundle** (the Docker Compose package that already exists) that an organisation runs on a server it controls, behind HTTPS. Offer a hosted single-organisation instance for customers who do not want to run anything. For regulated or air-gapped customers, offer the same containers as an offline package with checksums, and later a Helm chart for Kubernetes.

> **Why not an .exe**
> ASTRA is a server with three parts (a web studio, an API, a PostgreSQL database) used by many people and fed by other systems (SDKs, telemetry). An .exe would put a server on one person's laptop: no shared access, no backups, no patching, nothing for the SDKs to reach, and a bundled copy of Python and a database that you would then have to re-release for every security fix. It is also the form most likely to be blocked by endpoint protection.

What to say to a customer: "ASTRA runs on your own infrastructure as containers, behind your HTTPS and your sign-in, holds only statistics and event names, and updates from signed release versions that you approve."

Honest status: the application already has solid account, session and secret handling (section 5). The *deployment package* does not yet do HTTPS, security headers or single sign-on by itself; section 6 explains how to add them today and section 10 lists the product changes that would make a secure setup the default.

2. What has to be protected

Part	What it is	Exposure
Studio	nginx serving one web page; proxies `/api` to the backend on the same address	The only part users and SDKs reach
Backend (API)	FastAPI on port 8765; accounts, designs, contracts, findings, monitors	Internal only in the demo package
Database	PostgreSQL 16; the organisation map, designs, findings, hourly statistics, audit log, sealed secrets	Internal only; not published
Secrets	`POSTGRES_PASSWORD`, `ASTRA_SECRET_KEY` (encrypts stored repository tokens, identity-provider credentials and alert webhook addresses), API keys	In `.env` and in the database
Inbound feeds	SDK events and decisions, OpenTelemetry traces, from applications inside the organisation	Reach the studio address with a collector API key
Outbound calls	Git hosts, identity providers, alert webhooks (optional)	From the backend, to addresses the administrator configures

What ASTRA holds is deliberately thin. Production monitoring keeps hourly statistics, not records. Agent monitoring keeps event *names*, not content. Decision records are counted by role and drop names unless

compliance signs off. That limits the damage of a breach, and it is the strongest security argument ASTRA has. It still holds the organisation's AI inventory, designs, findings and evidence, which are sensitive: treat the database like a compliance system of record.

3. Deployment options

Option	How it works	Best for	Main risks and costs
A. Single server, Docker Compose (exists today)	One Linux VM; `deploy/docker-compose.demo.yml`; reverse proxy with a certificate in front	Start-ups, small and mid-size organisations, pilots, shared tests	One machine: backups and patching are the customer's job; no high availability
B. Customer's cloud account (VM or container service)	The same containers on the customer's AWS, Azure or GCP; managed PostgreSQL; load balancer with a managed certificate	Mid-size and larger organisations already on a cloud	More moving parts; networking and identity set-up per cloud
C. Kubernetes (Helm chart)	The three parts as deployments with ingress, secrets and a managed database	Large organisations with a platform team	A chart does not exist yet; backend must run as non-root first (section 10)
D. Air-gapped or on-premises, offline package	Container images exported as files with checksums; installed from an internal registry	Regulated sectors, public bodies, defence-adjacent	Updates are manual; no Ollama or internet assistants; release discipline needed
E. Hosted by you, one instance per customer	You run a separate stack for each customer behind your own domain	Customers with no infrastructure; a paid pilot	You become the processor: contracts, backups, incident duty. ASTRA is single-organisation per instance, so each customer needs its own stack
F. Windows or macOS .exe / installer	A packaged desktop program, or a wrapper that installs Docker	Not recommended; see below	Laptop becomes a server; patching and support burden; antivirus friction

3.1 The .exe question in detail

A real single executable would mean bundling Python 3.14, the backend's libraries, a web server and a database (an embedded PostgreSQL or a switch to SQLite, which the code does not support) into one file, then signing it with a code-signing certificate, shipping an updater, and re-releasing it for every dependency vulnerability. The gains are small: the only benefit is a one-click demo on a single laptop.

- **Security:** the program would run with the user's rights on an unmanaged machine; every release would need signing, and unsigned or self-extracting executables are flagged by endpoint protection.
- **Function:** the SDKs and telemetry feeds in the organisation's other systems need an address that stays up. A laptop does not provide one.
- **Support:** you would own an installer, an auto-updater and a Windows-specific bug list.

If customers still want a one-click experience, give them an *installer for the container bundle* instead: a small script or MSI that checks for Docker (or WSL2), creates `.env` with random secrets, starts the stack, creates the

administrator, and opens the browser. It keeps one code path, one set of patches and the same security model. For a single-machine evaluation it is as easy as an .exe and does not fork the product.

3.2 Recommended offering by customer type

Customer	Offer	Notes
Start-up or small business	A (single server) or E (hosted)	HTTPS through Caddy; sign-in by ASTRA accounts plus a VPN or access proxy
Mid-size organisation	B (their cloud) with managed PostgreSQL	Put it behind their single sign-on proxy; restrict by network
Large or regulated	C or D	Private registry, signed images, change approval for each update, penetration test before go-live
Universities, research pilots (IADS)	A on a university server	The current Docker package; add HTTPS and the university's sign-in proxy

4. Reference secure architecture



Figure 1. Only the edge is reachable from outside; the backend and the database are on a private container network.

- **One public address, one certificate.** Users and SDKs use the same HTTPS address. The studio proxies `/api` to the backend, so the page needs no cross-origin access and the API's origin list stays tight.
- **Private network between containers.** The compose file publishes only the studio's port; the API and the database have no published ports. Keep it that way.
- **Separate the edge from the studio container** where possible (Caddy, nginx, an application gateway or a cloud load balancer), so certificates, headers, rate limits and sign-in live in one place the security team already manages.
- **Separate environments.** Run a staging copy with synthetic data (the Helix example) for every update before production.

5. Security mechanisms: what ASTRA has, and what the deployment must add

5.1 Already in the application (verified in the code)

Area	Mechanism
Passwords	Stored only as a salted scrypt hash (n=16384). Changing a password is supported; an administrator can reset one.
Sessions	A random token per sign-in, kept server-side and stored only as a SHA-256 hash. Sent in an `HttpOnly`, `SameSite=Lax` cookie, marked `Secure` whenever the request arrives over HTTPS. 12-hour lifetime.

Area	Mechanism
Cross-site request protection	Every state-changing browser request must carry a custom header (X-ASTRA-CSRF), which another site cannot add without a cross-origin preflight; the API refuses such preflights. CORS allows only the origins in ASTRA_ALLOWED_ORIGINS.
Brute-force defence	Failed sign-ins are counted and a lockout answers with HTTP 429 for a window; each failure and denial is written to the audit log.
Roles	Admin, compliance, engineer and auditor for people; **collector** keys (ingest only) for applications. Sign-off and administration need compliance or admin; an auditor is read-only. A sign-off records the signed-in person: nobody can sign for someone else.
API keys	Created by an administrator for a role; a collector key can only send runtime data and read the contract summary it needs. Ingest is rate limited per key (429 with Retry-After).
Secrets at rest	Repository tokens, webhook secrets, identity-provider credentials and alert webhook addresses are encrypted with ASTRA_SECRET_KEY (Fernet). Several keys can be listed to rotate without losing data. They are never returned by the API. The platform refuses to store them if no key is set.
Outbound safety	Repository connections accept only https addresses and refuse hosts that resolve to private, loopback or link-local addresses unless the administrator sets ASTRA_REPO_ALLOW_PRIVATE_HOSTS=1. Connected code is read, never executed.
Audit	Sign-ins, denials, sign-offs, removals and halts are written to an audit log with actor, role and outcome; auditors can read it.
Data minimisation	Hourly statistics, event names, role-level decision counts; names are dropped unless compliance signs off, then they are one-way pseudonyms.
Container network	The demo compose file publishes only the studio port. The first person to open a new server can create the administrator, so the seed script claims it immediately.

5.2 Not in the package yet (the deployer must add it, or the product should)

Gap	Why it matters	Today's workaround
No HTTPS in the package	The compose file serves plain HTTP on one port; passwords and the session cookie would cross the network in clear text	Put Caddy, nginx with a certificate, or a cloud load balancer in front (section 6.1). Never expose the port directly
No security headers	The studio's nginx sets no HSTS, CSP, frame or content-type protections	Add them at the edge (section 6.2)
No single sign-on or MFA for staff	Sign-in is by ASTRA account and password. The identity-provider connector reads agent identities from Entra or Okta; it does not sign staff in	Put the studio behind an authenticating proxy (Entra, Okta, Google Workspace) or a VPN so MFA applies before ASTRA is reachable
Backend container runs as root	A flaw in the application would have more rights inside the container than needed	Add a non-root USER to the Dockerfile and a read-only filesystem (section 10)

Gap	Why it matters	Today's workaround
No edge rate limiting	Only sign-in and ingest are throttled in the application	Rate limit at the proxy or WAF
Secrets in a `.env` file	Readable by anyone with server access; lives in backups	File mode 600, a secrets manager, or Docker secrets; never in git
No CI pipeline in the repository	No automated tests, dependency checks, image scans or signing on each change	Add GitHub Actions (section 8)
No end-to-end encryption between the studio and the backend	Traffic on the private container network is plain HTTP	Acceptable on a single host; use a service mesh or TLS inside the cluster for higher assurance
Not covered by an external test	No independent penetration test or formal assessment yet	Commission one before a regulated customer goes live

6. Putting HTTPS and the other protections in place

6.1 HTTPS

Terminate TLS at the edge and forward to the studio's port on localhost or the private network. Choose by environment:

Environment	Approach
One server on the internet	**Caddy** in front (automatic certificates from Let's Encrypt, renewal, HTTP to HTTPS redirect). Three lines of configuration
One server on a corporate network	nginx or Caddy with a certificate from the organisation's **internal certificate authority**; clients already trust it
Cloud	The provider's load balancer or application gateway with a managed certificate; restrict the backend to the load balancer's address
Kubernetes	Ingress controller with cert-manager; network policies so only the studio talks to the backend and only the backend talks to the database
Air-gapped	Internal CA certificate; the same proxy; no public certificate authority involved

```
# Caddyfile: certificate, renewal and redirect are automatic
astra.example.org {
   reverse_proxy localhost:8080
   header {
      Strict-Transport-Security "max-age=31536000; includeSubDomains"
      X-Content-Type-Options nosniff
      X-Frame-Options DENY
      Referrer-Policy no-referrer
   }
}
```

- Allow **TLS 1.2 and 1.3** only; leave cipher choice to a current Mozilla "intermediate" profile or the platform default.
- Turn on **HSTS** only after HTTPS works everywhere, because browsers will refuse plain HTTP afterwards.
- **SDKs and OpenTelemetry exporters** must use the `https://` address. Where a customer requires it, issue client certificates (**mutual TLS**) at the edge for the collector path, in addition to the API key.

6.2 Headers and browser protections

Set these at the edge: `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY` (the studio should not be framed), `Referrer-Policy: no-referrer`. A `Content-Security-Policy` needs care: the studio is a single page with inline script and style, so a strict policy would require moving them to files first. Start with `frame-ancestors 'none'; base-uri 'none'` and tighten after that change.

6.3 Sign-in, MFA and network access

- **Best available today:** authenticate people at the edge before ASTRA is reached: an OIDC or SAML proxy (oauth2-proxy, the cloud's identity-aware proxy, Cloudflare Access, Entra application proxy) with MFA, plus ASTRA's own accounts behind it. Two layers, and the first one carries MFA and the organisation's joiner/leaver process.
- **Or** place the studio on the corporate network or a VPN, with an IP allow list, and publish only the collector path if applications live elsewhere.
- **Accounts:** create named accounts, give the least role needed (engineers cannot sign off; auditors are read-only), disable accounts when people leave, and use long passwords (the seed script requires at least 10 characters).
- **Collector keys:** one key per application, role collector only, stored in the application's secret store, rotated on a schedule and revoked on departure of the owner.

6.4 Secrets, keys and data at rest

- **`ASTRA_SECRET_KEY`**: generate with Fernet, store in a secrets manager as well as `.env`, back it up *separately from the database backup*. Rotate by listing the new key first and the old after it.
- **`POSTGRES_PASSWORD`**: long and random; the database is not published, so it is reachable only from the backend.
- **Disk or volume encryption** for the database volume and the backups (LUKS, BitLocker, or the cloud's managed encryption). A managed PostgreSQL service gives this by default.
- **Backups:** `pg_dump` on a schedule, encrypted, copied off the host, and **restore-tested**. Keep the `.env` and the secret key with them but not in the same place.
- **Retention:** monitors keep only what their retention setting says; set it with the organisation's data protection officer.

6.5 Hardening the host and the containers

- Patch the operating system; automatic security updates on; SSH by key only, no password, no root login.
- Firewall: allow 443 (and 80 for the redirect) only; deny everything else inbound.
- Docker: keep the engine current; do not mount the Docker socket; do not publish the API or database ports; drop capabilities and run the backend as a non-root user with a read-only filesystem once the Dockerfile supports it.
- Set `ASTRA_ALLOWED_ORIGINS` to the exact HTTPS address, and leave `ASTRA_REPO_ALLOW_PRIVATE_HOSTS` at 0 unless a Git server inside the network is intended.
- Do not install Ollama on an exposed host. The demo package leaves it out on purpose.

7. Monitoring and incident readiness

- **Logs:** collect the backend and proxy logs centrally; alert on repeated 401, 429 and 5xx responses and on sign-ins from unusual places.
- **Audit log:** export it on a schedule to the organisation's log store (SIEM), because a log on the same server can be lost with the server.

- **Health and capacity:** the stack has a health route and database health check; watch disk, memory and the size of the database.
- **Incident plan:** who revokes keys, who rotates `ASTRA_SECRET_KEY`, how to disable accounts, where the latest tested backup is, and who tells the data protection officer.
- **ASTRA's own alerts** (drift, findings, assurance gap) go to webhooks that are stored encrypted; keep those channels private.

8. Keeping up to date from GitHub, securely

The repository is the source of truth. The risk is not GitHub itself but an update arriving in production without anyone checking it. The pattern below keeps updates automatic to *prepare* and manual to *approve*.

8.1 Three ways to update, from simplest to most controlled

Model	How	Use when
Pull on the server	`git pull` of a *release tag*, then `docker compose up --build -d`. The existing package already works this way (database changes apply on their own and only add)	One server, small team, staging tested first
Prebuilt signed images from a registry	GitHub Actions builds the images on each release tag, scans them, signs them, and pushes to a **private** registry (GitHub Container Registry or the customer's). The server pulls the tagged images; it never builds	Most customers; recommended default
Offline package	A release produces image files plus a checksum and signature file; the customer verifies and loads them into an internal registry	Air-gapped and regulated customers

8.2 Controls on the GitHub side

- **Protect the main branch:** pull requests only, at least one review, required status checks, no force pushes, signed commits if the team can.
- **Releases from tags** (for example `v1.4.0`), created from `main` after checks pass, with release notes that say what changed and whether the database migrates. Customers pin to a tag, never to `latest` or a branch.
- **Automated checks on every pull request:** the existing test suite (`backend/tests`), dependency vulnerability scan (Dependabot or `pip-audit`), secret scanning (turn on GitHub secret scanning and push protection), and code scanning (CodeQL).
- **Image controls:** build from a pinned base image, scan with Trivy or Grype, produce an SBOM, and sign with cosign so the server can verify the image really came from your pipeline.
- **Least-privilege access:** servers that pull code use a read-only *deploy key* or a fine-grained token scoped to this repository, stored as a secret, never a personal token with wide rights. Turn on two-factor authentication for every maintainer.
- **No secrets in the repository.** The `.env` file is git-ignored; keep it that way. Treat the example organisation files as synthetic data only.

8.3 Controls on the server side

- **Staging first.** Apply every update to a staging stack with synthetic data; run the checks; then production.
- **Back up before updating.** Take a `pg_dump` immediately before, because a migration changes the database.

- **Approve, then apply.** A named person approves the release; the update is applied in a maintenance window. Avoid tools that automatically restart production on every new image.
- **Rollback:** keep the previous image tag; migrations only add, so the previous version still runs after an update. Restore the backup if a migration must be undone.
- **Verify after:** health route, sign-in, a test event from the SDK, the audit log recording the update.

8.4 A minimal release workflow (GitHub Actions)

```
name: release
on:
  push:
    tags: ["v*"]
permissions:
  contents: read
  packages: write
  id-token: write          # keyless signing with cosign
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Tests
        run: docker compose run --rm backend python -m tests.test_auth   # and the other test modules
      - uses: docker/login-action@v3
        with: { registry: ghcr.io, username: ${{ github.actor }}, password: ${{ secrets.GITHUB_TOKEN }} }
      - name: Build and push
        run: |
          docker build -t ghcr.io/khidawi/astra-backend:${GITHUB_REF_NAME} backend
          docker build -t ghcr.io/khidawi/astra-studio:${GITHUB_REF_NAME} studio
          docker push ghcr.io/khidawi/astra-backend:${GITHUB_REF_NAME}
          docker push ghcr.io/khidawi/astra-studio:${GITHUB_REF_NAME}
      - name: Scan
        uses: aquasecurity/trivy-action@0.24.0
        with: { image-ref: "ghcr.io/khidawi/astra-backend:${{ github.ref_name }}", exit-code: "1", severity:
"HIGH,CRITICAL" }
      - name: Sign
        uses: sigstore/cosign-installer@v3
        # then: cosign sign --yes ghcr.io/khidawi/astra-backend:${GITHUB_REF_NAME}
```

This is a starting sketch, not a tested pipeline. The compose file would reference the images by tag instead of building them, so production never compiles code.

9. Hardening checklist before go-live

Check	Done
HTTPS in front, plain HTTP redirected, TLS 1.2+ only; certificate renewal tested	☐
Only the edge is published; the API and database have no published ports; firewall allows 80/443 only	☐
HSTS, `nosniff`, frame protection and referrer policy set at the edge	☐
Administrator created by you before anyone else could reach the server; strong password	☐
Staff reach ASTRA only through MFA (authenticating proxy or VPN); accounts named, least role, leavers removed	☐
One collector key per application; stored in a secret store; rotation date set	☐
`.env` mode 600, not in git; `ASTRA_SECRET_KEY` and the database password stored in a secrets manager	☐

Check	Done
ASTRA_ALLOWED_ORIGINS set to the HTTPS address; private-host access left off unless needed	☐
Database volume and backups encrypted; a restore has been tested	☐
Logs and the audit log exported off the host; alerts on 401, 429 and 5xx	☐
Updates from a pinned release tag, tested on staging, backup taken first, approver named	☐
Operating system patched; SSH by key only; Docker engine current	☐
Data protection: purpose, retention and who may see decision-by-role analytics agreed with the DPO	☐
Independent security test completed (for regulated customers)	☐

10. Product changes that would make this the default

In priority order. None of them change what ASTRA does; they make a secure installation the easy one.

#	Change	Effect
1	A **production compose profile** with Caddy: automatic HTTPS, redirect and security headers, one variable for the domain	HTTPS stops being the customer's job
2	Run the backend as a **non-root user**; read-only filesystem; drop capabilities; add HEALTHCHECKs	Smaller blast radius
3	**CI pipeline**: tests, dependency and secret scanning, image scan, SBOM, signing; images published to a private registry from release tags	Updates become verifiable
4	**OIDC and SAML sign-in** for staff with MFA from the customer's identity provider, with role mapping	Removes the proxy workaround; joiner/leaver handled by the IdP
5	**Secrets from files** (Docker secrets or a mounted secret) and a startup check that refuses weak or missing values	No secrets in .env
6	A **Helm chart** with network policies and ingress TLS	Opens option C
7	**Installer script** for the container bundle (checks Docker, generates random secrets, claims the administrator, prints the URL)	The one-click experience without an .exe
8	Move inline script and style out of the single page and add a **Content-Security-Policy**	Closes the remaining browser-side gap
9	**Audit log export** (syslog or webhook) and an administrator-visible security status page	Easier SIEM integration and self-checks
10	An **independent penetration test** and a short security whitepaper for customers	Evidence for procurement

> **Suggested next step**
> Do items 1 to 3 first: they turn the current package into something you can hand to an organisation with HTTPS, a non-root container and verifiable updates, which covers most security questionnaires. Then add single sign-on (item 4), which is the question larger customers ask next. I can build items 1 and 2 in this repository and test them on a throwaway stack.